

5 - Funciones

Alejandro Veloz

aavelozb.github.io/py



Objetivos de la unidad

- Implementar **funciones** que resuelvan subproblemas dentro de un problema mayor, con una **interfaz bien definida**: parámetros y valor de retorno.
- Comprender la diferencia entre **print** y **return**.
- **Descomponer** un problema en subproblemas e **integrar** la solución final.
- Dar **seguimiento (ruteo)** a un programa que utiliza funciones.

Objetivos de la unidad

- Implementar **funciones** que resuelvan subproblemas dentro de un problema mayor, con una **interfaz bien definida**: parámetros y valor de retorno.
- Comprender la diferencia entre **print y return**.
- **Descomponer** un problema en subproblemas e **integrar** la solución final.
- Dar **seguimiento (ruteo)** a un programa que utiliza funciones.

Una función es una **caja**: entran datos, sale un resultado.

No es una instrucción nueva: es una forma de **organizar** lo que ya sabemos.

Ya usamos funciones

Desde la primera clase:

```
largo  = len(texto)
raiz   = sqrt(16)
codigo = ord('A')
n      = int(input('n: '))
valor  = round(3.14159, 2)
```

Ya usamos funciones

Desde la primera clase:

```
largo  = len(texto)
raiz    = sqrt(16)
codigo = ord('A')
n       = int(input('n: '))
valor  = round(3.14159, 2)
```

En todos los casos:

- le **entregamos** datos entre paréntesis (los *argumentos*),
- la función **hace un trabajo** que no nos interesa ver por dentro,
- y nos **devuelve** un resultado que guardamos o usamos.

Ya usamos funciones

Desde la primera clase:

```
largo  = len(texto)
raiz    = sqrt(16)
codigo = ord('A')
n       = int(input('n: '))
valor  = round(3.14159, 2)
```

En todos los casos:

- le **entregamos** datos entre paréntesis (los *argumentos*),
- la función **hace un trabajo** que no nos interesa ver por dentro,
- y nos **devuelve** un resultado que guardamos o usamos.

Ahora vamos a escribir las nuestras.

El problema

Calcular la distancia entre tres pares de puntos:

```
d1 = sqrt((x1-x2)**2 + (y1-y2)**2)
```

```
d2 = sqrt((x3-x4)**2 + (y3-y4)**2)
```

```
d3 = sqrt((x5-x6)**2 + (y5-y6)**2)
```


El problema

Calcular la distancia entre tres pares de puntos:

```
d1 = sqrt((x1-x2)**2 + (y1-y2)**2)
```

```
d2 = sqrt((x3-x4)**2 + (y3-y4)**2)
```

```
d3 = sqrt((x5-x6)**2 + (y5-y6)**2)
```

Problemas:

- La fórmula está escrita **tres veces**.
- Si hay un error, hay que corregirlo **tres veces**.
- El programa no dice qué **significa** esa expresión.

El problema

Calcular la distancia entre tres pares de puntos:

```
d1 = sqrt((x1-x2)**2 + (y1-y2)**2)
```

```
d2 = sqrt((x3-x4)**2 + (y3-y4)**2)
```

```
d3 = sqrt((x5-x6)**2 + (y5-y6)**2)
```

Problemas:

- La fórmula está escrita **tres veces**.
- Si hay un error, hay que corregirlo **tres veces**.
- El programa no dice qué **significa** esa expresión.

Lo que queremos:

```
d1 = distancia(x1, y1, x2, y2)
```

```
d2 = distancia(x3, y3, x4, y4)
```

```
d3 = distancia(x5, y5, x6, y6)
```

Escribir la fórmula **una vez** y darle un **nombre**.

Definir y usar una función

Definición: def

```
def nombre(parametro1, parametro2):  
    instruccion 1  
    instruccion 2  
return resultado
```

Definición: def

```
def nombre(parametro1, parametro2):  
    instruccion 1  
    instruccion 2  
    return resultado
```

- def inicia la **definición**. Termina con **dos puntos**.
- Entre paréntesis van los **parámetros**: los datos que la función necesita para trabajar.
- El **cuerpo** va indentado, igual que en if y while.
- return entrega el **resultado** a quien llamó a la función.

Definición: def

```
def nombre(parametro1, parametro2):  
    instruccion 1  
    instruccion 2  
    return resultado
```

- def inicia la **definición**. Termina con **dos puntos**.
- Entre paréntesis van los **parámetros**: los datos que la función necesita para trabajar.
- El **cuerpo** va indentado, igual que en if y while.
- return entrega el **resultado** a quien llamó a la función.

Definir una función no la ejecuta. Sólo la deja escrita.

Definición y llamada

```
def sumar(a, b):           # definición
    suma = a + b
    return suma

num1 = 2
num2 = 3
s = sumar(num1, num2)      # llamada (invocación)
print(s)                   # 5
```

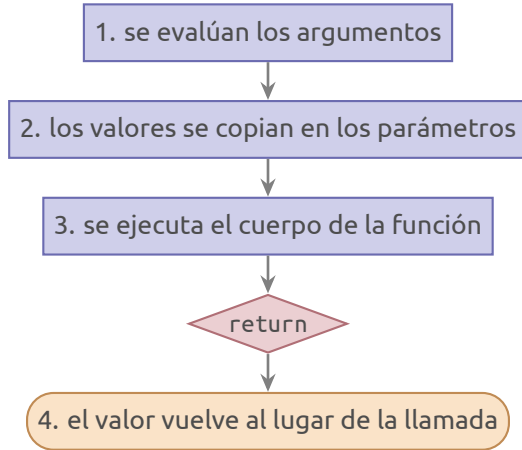
Definición y llamada

```
def sumar(a, b):           # definición
    suma = a + b
    return suma

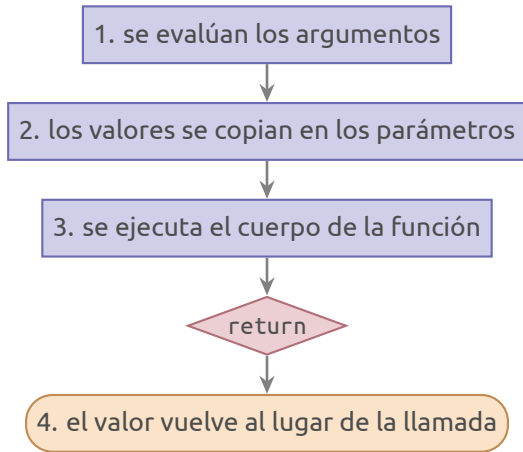
num1 = 2
num2 = 3
s = sumar(num1, num2)     # llamada (invocación)
print(s)                  # 5
```

- a y b son los **parámetros** (nombres en la definición).
- num1 y num2 son los **argumentos** (valores en la llamada).
- Al llamar, a toma el valor 2 y b toma el valor 3.

¿Qué pasa al llamar una función?



¿Qué pasa al llamar una función?



El programa principal se **detiene** mientras la función trabaja, y continúa exactamente donde estaba.

Ejemplo completo: distancia

```
from math import sqrt

def distancia(x1, y1, x2, y2):
    t1 = (x1-x2)**2
    t2 = (y1-y2)**2
    d = sqrt(t1+t2)
    return d

a = float(input('x del primer punto: '))
b = float(input('y del primer punto: '))
c = float(input('x del segundo punto: '))
e = float(input('y del segundo punto: '))
dist = distancia(a, b, c, e)
print('La distancia es:', dist)
```

Dónde se escriben las funciones

```
from math import sqrt      # 1. importaciones

def distancia(x1, y1, x2, y2):  # 2. definiciones
    ...

def area(a, b, angulo):
    ...

a = float(input('x: '))      # 3. programa principal
...
```

Dónde se escriben las funciones

```
from math import sqrt      # 1. importaciones

def distancia(x1, y1, x2, y2):  # 2. definiciones
    ...

def area(a, b, angulo):
    ...

a = float(input('x: '))      # 3. programa principal
...
```

Las funciones se definen **antes de usarlas**. Por convención, todas juntas al comienzo del archivo, después de los import.

print vs return

print vs return

¿Qué imprime cada programa?

```
def mifuncion(x):  
    y = 2*x+3  
    print(y)  
  
a = 5  
print(a, mifuncion(a))
```

print vs return

¿Qué imprime cada programa?

```
def mifuncion(x):  
    y = 2*x+3  
    print(y)
```

```
a = 5  
print(a, mifuncion(a))  
13  
5 None
```

```
def mifuncion(x):  
    y = 2*x+3  
    return y
```

```
a = 5  
print(a, mifuncion(a))
```


print vs return

¿Qué imprime cada programa?

```
def mifuncion(x):  
    y = 2*x+3  
    print(y)
```

```
a = 5  
print(a, mifuncion(a))  
13  
5 None
```

```
def mifuncion(x):  
    y = 2*x+3  
    return y
```

```
a = 5  
print(a, mifuncion(a))  
5 13
```

print vs return

¿Qué imprime cada programa?

```
def mifuncion(x):  
    y = 2*x+3  
    print(y)
```

```
a = 5  
print(a, mifuncion(a))  
13  
5 None
```

```
def mifuncion(x):  
    y = 2*x+3  
    return y
```

```
a = 5  
print(a, mifuncion(a))  
5 13
```

Una función sin return retorna None: "nada". Se puede imprimir, pero no sirve para calcular.

print vs return

	print(y)	return y
¿Qué hace?	Muestra y en pantalla	Entrega y a quien llamó
¿El valor sirve después?	No, se perdió	Sí, se puede guardar y usar
¿Quién lo ve?	La persona usuaria	El programa

print vs return

	print(y)	return y
¿Qué hace?	Muestra y en pantalla	Entrega y a quien llamó
¿El valor sirve después?	No, se perdió	Sí, se puede guardar y usar
¿Quién lo ve?	La persona usuaria	El programa

Regla del curso: las funciones **no imprimen**.

La función calcula y retorna; el programa principal decide qué mostrar.

return también termina la función

```
def es_divisible(a, b):  
    if a % b == 0:  
        return True  
    return False
```

return también termina la función

```
def es_divisible(a, b):  
    if a % b == 0:  
        return True  
    return False
```

¿Falta el else? **No**. Si $a \% b == 0$, el primer return entrega True y la función **termina ahí mismo**: la última línea nunca se alcanza.

return también termina la función

```
def es_divisible(a, b):  
    if a % b == 0:  
        return True  
    return False
```

¿Falta el else? **No**. Si $a \% b == 0$, el primer return entrega True y la función **termina ahí mismo**: la última línea nunca se alcanza.

return tiene dos roles: entregar el resultado **y** volver inmediatamente al lugar de la llamada.

Todo lo que se escriba después de un return ejecutado es código muerto.

Retornar directamente una condición

Este patrón aparece mucho:

```
def es_par(n):  
    if n % 2 == 0:  
        return True  
    else:  
        return False
```


Retornar directamente una condición

Este patrón aparece mucho:

```
def es_par(n):  
    if n % 2 == 0:  
        return True  
    else:  
        return False
```

Es correcto, pero `n % 2 == 0` ya es un valor booleano:

```
def es_par(n):  
    return n % 2 == 0
```

Retornar directamente una condición

Este patrón aparece mucho:

```
def es_par(n):  
    if n % 2 == 0:  
        return True  
    else:  
        return False
```

Es correcto, pero `n % 2 == 0` **ya es** un valor booleano:

```
def es_par(n):  
    return n % 2 == 0
```

Mismo resultado, tres líneas menos.

Parámetros

El orden importa

Complete el programa para calcular la pendiente de la recta que pasa por (0, 0) y (1, 1):

```
def pendiente(x1, x2, y1, y2):  
    m = (y2-y1)/(x2-x1)  
    return m
```

```
_____ pendiente(_____)  
print(p)
```

El orden importa

Complete el programa para calcular la pendiente de la recta que pasa por (0, 0) y (1, 1):

```
def pendiente(x1, x2, y1, y2):  
    m = (y2-y1)/(x2-x1)  
    return m
```

```
_____ pendiente(_____)  
print(p)
```

```
p = pendiente(0, 1, 0, 1)
```

Ojo con el orden de esta definición: los parámetros son x1, x2, y1, y2 y no x1, y1, x2, y2. Python no avisa: entrega un resultado equivocado.

La cantidad también importa

```
def sumar(a, b):  
    return a + b  
  
print(sumar(2))           # TypeError: missing 1 argument  
print(sumar(2, 3, 4))     # TypeError: takes 2 arguments
```

La cantidad también importa

```
def sumar(a, b):  
    return a + b  
  
print(sumar(2))           # TypeError: missing 1 argument  
print(sumar(2, 3, 4))     # TypeError: takes 2 arguments
```

- La cantidad de argumentos debe coincidir **exactamente** con la de parámetros.
- La correspondencia es **por posición**: el primer argumento va al primer parámetro.

La cantidad también importa

```
def sumar(a, b):  
    return a + b  
  
print(sumar(2))          # TypeError: missing 1 argument  
print(sumar(2, 3, 4))    # TypeError: takes 2 arguments
```

- La cantidad de argumentos debe coincidir **exactamente** con la de parámetros.
- La correspondencia es **por posición**: el primer argumento va al primer parámetro.

La lista de parámetros y el valor de retorno son la **interfaz** de la función:

Lo único que hay que saber para usarla.

Los parámetros se pasan por valor

```
def duplicar(n):
```

```
    n = n * 2
```

```
    return n
```

```
x = 5
```

```
y = duplicar(x)
```

```
print(x, y)      # 5 10
```

Los parámetros se pasan por valor

```
def duplicar(n):  
    n = n * 2  
    return n  
  
x = 5  
y = duplicar(x)  
print(x, y)      # 5 10
```

Dentro de la función, `n` es una **copia**: cambiarla no afecta a `x`.

Los parámetros se pasan por valor

```
def duplicar(n):  
    n = n * 2  
    return n  
  
x = 5  
y = duplicar(x)  
print(x, y)      # 5 10
```

Dentro de la función, `n` es una **copia**: cambiarla no afecta a `x`.

La única forma de sacar un resultado de una función es `return`.

Variables locales y globales

Variables locales

Las variables creadas dentro de una función (**incluidos los parámetros**) sólo existen mientras la función se ejecuta.

```
def sumar(a, b):  
    suma = a + b  
    return suma
```

```
s = sumar(2, 3)  
print(suma)      # NameError: name 'suma' is not defined
```

Variables locales

Las variables creadas dentro de una función (**incluidos los parámetros**) sólo existen mientras la función se ejecuta.

```
def sumar(a, b):  
    suma = a + b  
    return suma  
  
s = sumar(2, 3)  
print(suma)      # NameError: name 'suma' is not defined
```

Esto es una ventaja: podemos usar `i`, `suma` o `cont` dentro de una función sin preocuparnos de lo que ocurre afuera.

Mismo nombre, distinta variable

¿Qué imprime este programa?

```
def es_divisible(a, b):  
    if a % b == 0:  
        return True  
    else:  
        return False  
  
a = 5  
b = 10  
if es_divisible(b, a):  
    print(b, "es divisible por", a)  
else:  
    print(b, "no es divisible por", a)
```

Mismo nombre, distinta variable

¿Qué imprime este programa?

```
def es_divisible(a, b):  
    if a % b == 0:  
        return True  
    else:  
        return False  
  
a = 5  
b = 10  
if es_divisible(b, a):  
    print(b, "es divisible por", a)  
else:  
    print(b, "no es divisible por", a)
```

10 es divisible por 5

Mismo nombre, distinta variable

- Las variables a y b del **programa** son distintas de los parámetros a y b de la **función**.
- En la llamada `es_divisible(b, a)` se pasa `b=10` como primer argumento, así que dentro de la función el parámetro a vale 10.

Mismo nombre, distinta variable

- Las variables a y b del **programa** son distintas de los parámetros a y b de la **función**.
- En la llamada `es_divisible(b, a)` se pasa `b=10` como primer argumento, así que dentro de la función el parámetro a vale 10.

Coincidir los nombres confunde, pero no es un error: lo que manda es la **posición** de los argumentos.

Mismo nombre, distinta variable

- Las variables a y b del **programa** son distintas de los parámetros a y b de la **función**.
- En la llamada `es_divisible(b, a)` se pasa `b=10` como primer argumento, así que dentro de la función el parámetro a vale 10.

Coincidir los nombres confunde, pero no es un error: lo que manda es la **posición** de los argumentos.

Todo lo que una función necesita se le pasa como **parámetro**.

No usamos variables globales dentro de las funciones.

Descomponer un problema

Una función, un subproblema

Una buena función:

- resuelve **una** cosa, y su nombre lo dice (es_primo, area_triangulo, digito_verificador);
- recibe **todo** lo que necesita por parámetros;
- retorna **un** valor;
- no lee (input) ni imprime (print).

Una función, un subproblema

Una buena función:

- resuelve **una** cosa, y su nombre lo dice (es_primo, area_triangulo, digito_verificador);
- recibe **todo** lo que necesita por parámetros;
- retorna **un** valor;
- no lee (input) ni imprime (print).

Si al describir una función hay que usar un “y”, probablemente son dos funciones.

Funciones que usan otras funciones

El área de un triángulo a partir de dos lados y el ángulo entre ellos:

$$A = \frac{1}{2} ab \sin(\theta)$$

```
from math import sin, pi

def grados_a_radianes(grados):
    return grados * pi / 180

def area_triangulo(a, b, angulo):
    return 0.5 * a * b * sin(grados_a_radianes(angulo))

print(area_triangulo(1, 2, 30))    # 0.5
```

Funciones que usan otras funciones

El área de un triángulo a partir de dos lados y el ángulo entre ellos:

$$A = \frac{1}{2} ab \sin(\theta)$$

```
from math import sin, pi

def grados_a_radianes(grados):
    return grados * pi / 180

def area_triangulo(a, b, angulo):
    return 0.5 * a * b * sin(grados_a_radianes(angulo))

print(area_triangulo(1, 2, 30))    # 0.5
```

La función `area_triangulo` no necesita saber cómo se convierten los grados.

Estrategia: de arriba hacia abajo

Primero se escribe el programa principal **suponiendo que las funciones ya existen:**

```
horas = int(input('Horas trabajadas: '))
valor = int(input('Valor hora: '))

base = salario_base(horas, valor)
imponible = salario_imponible(base)
impuesto = impuesto_unico(imponible)

print('Salario base:', base)
print('Líquido:', imponible - impuesto)
```

Estrategia: de arriba hacia abajo

Primero se escribe el programa principal **suponiendo que las funciones ya existen**:

```
horas = int(input('Horas trabajadas: '))
valor = int(input('Valor hora: '))

base = salario_base(horas, valor)
imponible = salario_imponible(base)
impuesto = impuesto_unico(imponible)

print('Salario base:', base)
print('Líquido:', imponible - impuesto)
```

Después se implementa cada función por separado. **Cada una se puede probar sola.**

Ruteo

Ruteo con funciones

Igual que antes, pero ahora hay **dos zonas de variables**: las del programa y las de cada función.

- Se dibuja una tabla para el **programa principal** y una para **cada función**.
- Al llamar, se anotan los valores que reciben los parámetros.
- Al ejecutar return, el valor vuelve a la expresión donde estaba la llamada.
- Cuando la función termina, sus variables **desaparecen**.

Ruteo con funciones

Igual que antes, pero ahora hay **dos zonas de variables**: las del programa y las de cada función.

- Se dibuja una tabla para el **programa principal** y una para **cada función**.
- Al llamar, se anotan los valores que reciben los parámetros.
- Al ejecutar return, el valor vuelve a la expresión donde estaba la llamada.
- Cuando la función termina, sus variables **desaparecen**.

Si la misma función se llama dos veces, se rutea dos veces.

Ruteo: coeficiente binomial

```
def factorial(n):  
    f = 1  
    i = 2  
    while i <= n:  
        f *= i  
        i += 1  
    return f  
  
def coef_binomial(n, k):  
    a = factorial(n)  
    b = factorial(k)  
    c = factorial(n-k)  
    return a/(b*c)  
  
coef = coef_binomial(3, 2)  
print(coef)
```

factorial se llama **tres veces**: con 3, con 2 y con 1.

factorial			
n	f	i	ret

Ruteo: coeficiente binomial

```
def factorial(n):  
    f = 1  
    i = 2  
    while i <= n:  
        f *= i  
        i += 1  
    return f  
  
def coef_binomial(n, k):  
    a = factorial(n)  
    b = factorial(k)  
    c = factorial(n-k)  
    return a/(b*c)  
  
coef = coef_binomial(3, 2)  
print(coef)
```

factorial se llama **tres veces**: con 3, con 2 y con 1.

factorial			
n	f	i	ret

Imprime 3.0

Ruteo: funciones y strings

```
def f(p):  
    if '-' in p:  
        x = p[1]  
    else:  
        x = p[3]  
    return x
```

```
q = 'HBRU 93'  
s = f(q)  
q = 'XS-7122'  
c = f(q)  
s = s + c  
q = 'CM-4513'  
c = f(q)  
s = s + c  
print(s)
```

¿Qué imprime?

Ruteo: funciones y strings

```
def f(p):  
    if '-' in p:  
        x = p[1]  
    else:  
        x = p[3]  
    return x
```

```
q = 'HBRU 93'  
s = f(q)  
q = 'XS-7122'  
c = f(q)  
s = s + c  
q = 'CM-4513'  
c = f(q)  
s = s + c  
print(s)
```

¿Qué imprime?

'USM'

'HBRU 93' no tiene guión $\rightarrow p[3] = 'U'$

'XS-7122' tiene guión $\rightarrow p[1] = 'S'$

'CM-4513' tiene guión $\rightarrow p[1] = 'M'$

Errores frecuentes

Encuentre los errores

La función debe retornar el máximo común divisor de a y b.

```
def euclides(a, b):  
    a = int(input('a: '))  
    b = int(input('b: '))  
    while b != 0:  
        resto = a % b  
        a = b  
        b = resto  
    print(a)
```

Encuentre los errores

La función debe retornar el máximo común divisor de a y b.

```
def euclides(a, b):  
    a = int(input('a: '))  
    b = int(input('b: '))  
    while b != 0:  
        resto = a % b  
        a = b  
        b = resto  
    print(a)
```

- (1) Lee con input los valores que **ya venían** en los parámetros: los pisa.
- (2) Usa print en vez de return: el resultado no se puede usar.

Encuentre el error

```
def cuadrado(n):
```

```
    n ** 2
```

```
print(cuadrado(4))
```

Encuentre el error

```
def cuadrado(n):
```

```
    n ** 2
```

```
print(cuadrado(4))
```

Imprime None: se calcula $n ** 2$ y se descarta, porque falta el return.

```
def cuadrado(n):
```

```
    return n ** 2
```

Encuentre el error

```
def duplicar(n):
```

```
    return n * 2
```

```
x = 5
```

```
duplicar(x)
```

```
print(x)      # 5
```

Encuentre el error

```
def duplicar(n):
```

```
    return n * 2
```

```
x = 5
```

```
duplicar(x)
```

```
print(x)      # 5
```

La llamada no sirve de nada: el valor retornado se pierde porque no se asigna ni se usa.

```
x = duplicar(x)
```


Encuentre el error

```
def promedio(a, b, c):  
    return (a + b + c) / 3
```

```
p = promedio(4, 5)
```

Encuentre el error

```
def promedio(a, b, c):  
    return (a + b + c) / 3
```

```
p = promedio(4, 5)
```

TypeError: la función espera **tres** argumentos y se le pasaron dos.

Encuentre el error

```
def promedio(a, b, c):  
    return (a + b + c) / 3  
  
p = promedio(4, 5)
```

TypeError: la función espera **tres** argumentos y se le pasaron dos.

Antes de llamar a una función, revise su interfaz: cuántos parámetros son, en qué orden, y qué retorna.

Lo que usamos en esta unidad

SÍ

- Definición con def.
- Parámetros y valor de retorno.
- Llamada o invocación.
- Diferencia entre print() y return.
- Variables locales y su alcance.
- Descomposición en subproblemas y reutilización de código.
- Ruteo de programas con funciones.

NO

- print o input dentro de una función.
- Funciones sin valor de retorno.
- Retornar varios valores.
- Variables globales dentro de funciones.
- Recursión.
- Parámetros por defecto.

Toda función retorna **un** valor, y todo lo que necesita se le pasa como parámetro.

Ejercicios

Ejercicios de calentamiento

(a) Escriba una función que convierta **grados en radianes**. Recuerde que π radianes equivalen a 180 grados.

(b) Escriba una función que retorne la **cantidad de soluciones reales** de $ax^2 + bx + c = 0$. El discriminante es $b^2 - 4ac$: si es positivo hay 2 soluciones, si es cero hay 1, y si es negativo hay 0.

(c) Escriba una función que determine si un entero positivo es **perfecto**, retornando True o False. Un número es perfecto si es igual a la suma de sus divisores propios: $28 = 1 + 2 + 4 + 7 + 14$.

Ejercicio: palíndromos

Escriba una función que determine si un número entero es un **palíndromo** considerando sus dígitos, retornando True o False.

```
>>> es_palindromo(12321)
True
>>> es_palindromo(1221)
True
```

```
>>> es_palindromo(123)
False
>>> es_palindromo(7)
True
```

Restricción: no use strings. Trabaje con % y //, como en la unidad de ciclos.

Ejercicio: dígito verificador del RUT

El dígito verificador de un RUT se calcula así:

1. Multiplicar cada dígito, **de derecha a izquierda**, por 2, 3, 4, 5, 6, 7, y volver a 2.
2. Sumar todos los productos.
3. Calcular $11 - \text{suma} \% 11$. Si da 11 el dígito es 0; si da 10 es 'k'.

(a) Escriba `suma_productos(rut)`, que retorna la suma del paso 2, y `digito_verificador(rut)`.

(b) Escriba el programa que la usa:

```
Ingresar tu RUT: 15136120
15136120-k
```


Ejercicio: día de la semana

(a) Escriba una función que, dada una fecha, retorne el **día de la semana** correspondiente.

(b) Escriba un programa que solicite el nombre y la fecha de nacimiento de la persona. La fecha se ingresa como **string** con / como separador y siempre 10 caracteres (dd/mm/aaaa). El programa debe saludar indicando el día en que nació.

```
Cómo te llamas?: Bob
```

```
Cuál es tu fecha de nacimiento (dd/mm/aaaa)?: 01/06/2001
```

```
Hola Bob, tú naciste un viernes.
```

Ejercicio: restricción vehicular

Existen patentes de dos formatos: 4 letras, espacio y 2 dígitos ('CRTJ 32'), o 2 letras, guión y 4 dígitos ('RX-2134').

Catalíticos (última letra):

- Lunes: menor o igual a 'G'.
- Miércoles: mayor que 'G' y menor o igual a 'N'.
- Viernes: mayor que 'N'.

No catalíticos (último dígito):

- Lunes: 0, 1, 2 o 3.
- Miércoles: 4, 5 o 6.
- Viernes: 7, 8 o 9.

Escriba `tiene_restriccion(patente, catalitico, dia)`, que retorna `True` si corresponde cursar un parte.

```
>>> tiene_restriccion('CRTJ 32', True, 'LUNES')    # False
>>> tiene_restriccion('ZZ-9999', True, 'VIERNES')   # True
>>> tiene_restriccion('RX-2134', False, 'MIERCOLES') # True
```

Ejercicio: hashtags

Un *hashtag* es una palabra dentro de un mensaje que comienza con #.

(a) Escriba `extraer_temas(tweet)`, que retorna un string con los hashtags del mensaje, **sin** el # y separados por espacios. Si no hay hashtags, retorna el string vacío.

```
>>> print(extraer_temas('Hoy saqué a pasear a mis perros'))  
  
>>> print(extraer_temas('Lo mejor de la pandemia #covid19 son las clases #usm'))  
covid19 usm
```

(b) Escriba `aparece_como_hashtag(tweet, palabra)`, que retorna `True` si la palabra aparece **como hashtag** en el tweet.

```
>>> print(aparece_como_hashtag('Saqué mis perros #doglover', 'doglover'))  
True  
>>> print(aparece_como_hashtag('Saqué mis perros doglover', 'doglover'))  
False
```

Ejercicio: crédito hipotecario

La cuota mensual m de un crédito por un capital h , a n años, con interés compuesto de $i\%$ anual, es:

$$m = \frac{h \cdot r}{1 - (1 + r)^{-12n}}, \quad r = \frac{i}{100 \cdot 12}$$

Escriba, **una función por cada punto**:

1. `cuota_mensual(h, n, i)`, redondeada a 2 decimales.
2. `monto_total(h, n, i)`: el total pagado.
3. `intereses_pagados(h, n, i)`.
4. `porcentaje_intereses(h, n, i)`.
5. Un programa que, para h e i dados, muestre estos valores para un rango de plazos.

Para \$50.000.000, 15 años y 4.75%:

```
cuota:      388915.96
total:      70004872.8
intereses:  20004872.8
porcentaje:  40.01
```

Note cómo (2), (3) y (4) se apoyan en la función (1).

Ejercicio: Deuman con funciones

Recuerde la encuesta Deuman: cada persona vota con un entero cuyos dígitos son notas de 1 a 7, y su calificación es el **promedio de los dígitos**.

(a) Escriba la función `calificacion(voto)`, que recibe un voto y retorna el promedio de sus dígitos redondeado a 2 decimales.

(b) Reescriba el programa de la unidad de ciclos (que leía votos hasta el 0 y calculaba máximo, mínimo y promedio) **usando esta función**, en vez del ciclo anidado.

Ejercicio: Deuman con funciones

Recuerde la encuesta Deuman: cada persona vota con un entero cuyos dígitos son notas de 1 a 7, y su calificación es el **promedio de los dígitos**.

(a) Escriba la función `calificacion(voto)`, que recibe un voto y retorna el promedio de sus dígitos redondeado a 2 decimales.

(b) Reescriba el programa de la unidad de ciclos (que leía votos hasta el 0 y calculaba máximo, mínimo y promedio) **usando esta función**, en vez del ciclo anidado.

El ciclo anidado desaparece: queda un ciclo que llama a una función.

Ejercicio: liquidación de sueldo

El salario base se calcula a partir de las horas trabajadas en el mes y del monto por hora del empleado. Las horas extra (sobre 44) se pagan a 1.5 veces el valor de la hora regular, con un máximo de 40 horas extra mensuales.

Al salario base se le descuenta salud (7%) y previsión (10% de ahorro más 1.27% de comisión). Sobre el resultado se aplica el Impuesto Único de Segunda Categoría:

Hasta	Tasa	Descontar
\$ 680.022	exento	
\$ 1.511.160	4%	\$ 27.200,88
\$ 2.518.600	8%	\$ 87.647,28
\$ 3.526.040	13,5%	\$ 226.170,28
y más	...	...

Escriba un programa que calcule el salario que recibirá un empleado al término del mes, mostrando el detalle de lo ganado y lo descontado. [Descomponga en funciones antes de escribir código.](#)